

part #2 of lecture 8: distributed training:

def data-parallelism → sharding strategy is to keep each rank gets a slice of
 ⇒ losses are different across ranks [computed on local data] the data.
 ⇒ Gradients are all-reduced to be the same across ranks
 ⇒ therefore, parameters remain the same across ranks.

~~def tensor-parallelism~~:

def generate-data():

def data-parallelism-main():

Get the slice of data for this rank

[each rank should load its own data]

create MLP params, each rank

[each rank has all params]

forward pass

update

backward pass

weights

sync gradients across workers

def tensor-parallelism →

sharding strategy: each rank gets part of each layer, transfer all data / activations

y

data = generate-sample-data()

spawn (tensor-parallelism-main, world-size=4, data=data, num-layers=4)

def tensor-parallelism-main():

layer 3

layer 2

layer 1

layer 0



data sharding

spawn: creates/launches world-size child process;
 2. each process get its own process rank(0, 1, ..., world-1);
 3. each process run main function

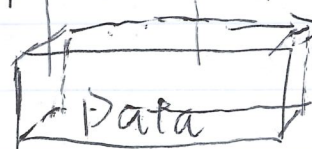
spawn: is a process launcher
 ⇒ it starts multiple worker process that each run a target function, e.g. here (tensor-parallelism-main)

layer 3

layer 2

layer 1

layer 0



layer sharding